

# 1. Table of Contents

- **Installation**
- **Uninstallation**
- **Quickstart**

1. Overview
  2. Key Features
  3. Installation & Setup
    - 3.1 Fixing the MemoryPack Error
  4. Basic Workflow Scene GameObject
    - 4.1 Registering an Instantiated Prefab
  5. Using Crystal Save Components
  6. Instantiate a Saveable Prefab (Remember Prefab) at runtime & Pools
  7. Targeted Restore - Restoring a single GameObject
  8. Restoring Destroyed GameObjects
  9. Scene Transitions & Keep Across Scenes
  10. User Settings Manager
  11. Working With GC2 Integration
  12. Save Slots and Screenshots
  13. Auto-Migration & Versioning
  14. Limitations & Caveats
  15. API References
  16. Conclusion
  17. Credits
-

## Installation:

### Installation (first import)

---

When you open the project after importing Crystal Save the editor will show a prompt:

**Crystal Save needs MemoryPack (1.21.4) and Unity Editor Coroutines.**

**Install now?** [Install] [Cancel]

- Click **\*\*Install\*\*** to download  $\approx 2$  MB (MemoryPack Core & Generator, System.Collections.Immutable / Unsafe) and add the official Editor Coroutines package from the Unity Registry.
- Click **\*\*Cancel\*\*** to skip. Crystal Save will fall back to Unity's built-in serialization; install the packages later via **\_Tools > Crystal Save > Install Dependencies\_ or your package manager.**

## Uninstallation:

Navigate in the Menu to Tools/Crystal Save/Project/Uninstall Crystal Save

**Please Note:** Uninstallation **will take a minimum of 2 minutes** because it scans the whole project for Crystal Save Components and removes them from the prefabs and scenes. The Editor does not react to input during uninstallation. Please do not cancel the process and wait.

## Quick Start:

### No Scene Preparation, Prefab Registrations required

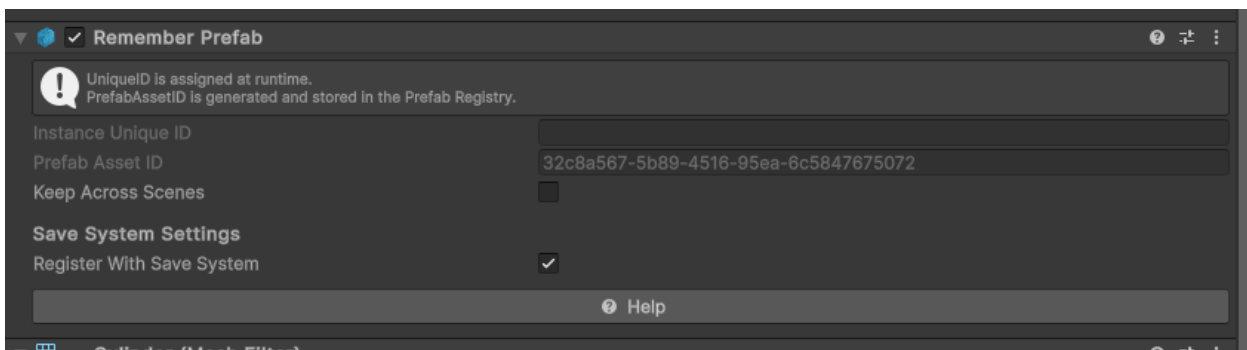
Open the Crystal Save Setting, **Menu: Crystal Save/Settings/Crystal Save Settings**. This will auto-install empty setting files.

**If you plan to use the Demo Scenes:** Install the Default Setting → Menu: Tools/Crystal Save/Settings/Install Default Settings

**Save/Load Prefabs:** Put the **Remember Prefab component on your prefab. Done!**

Saving and Loading will now restore this Prefab. Easier than ever!

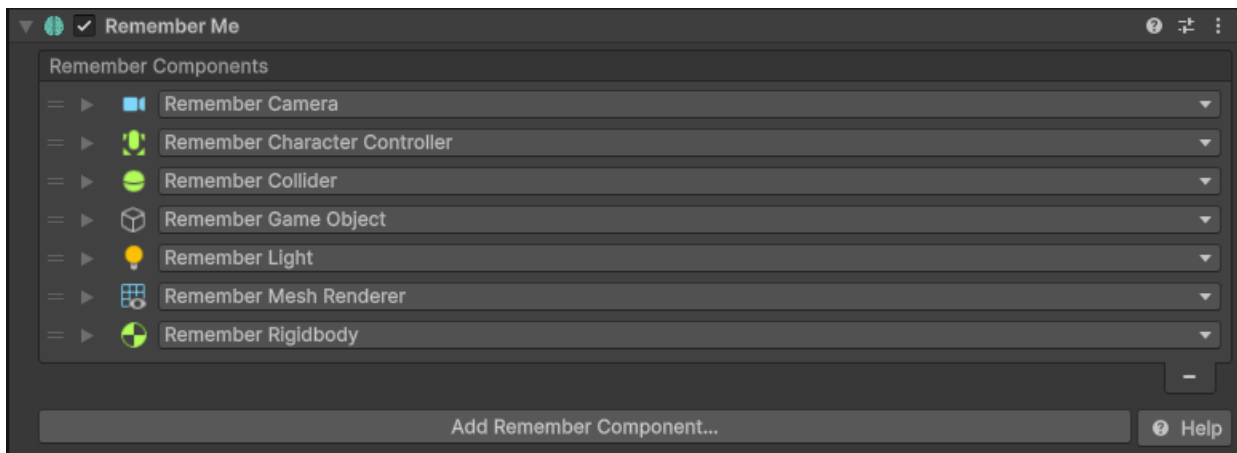
**Save/Load Scene GameObject:** Put the **Remember Component on your prefab. Add a Remember**



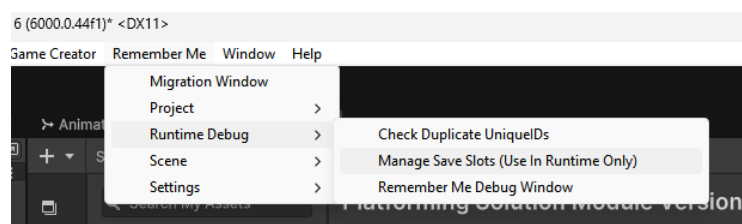
**Module like Remember Transform** to save/load the Transform Component. Done!

Saving and Loading will now restore this GameObject's Transform.

**During Runtime** go to the Menu: **Tools/Crystal Save/Runtime Debug/Manage Save Slots** to **save** and



**load** your game



# 1. Overview

**Crystal Save** is a Save Framework for Unity, focusing on ease of use, performance and reliability. By leveraging **MemoryPack** to serialize data in binary, it achieves high efficiency and small file sizes. It also integrates with **Game Creator 2 (Optional Module)**, allowing you to save GC2 Variables, custom Items (e.g., bag content), and more.

## Why Binary Serialization?

- **Performance & Size:** Storing data in bytes instead of JSON greatly reduces file size—crucial for mobile platforms where storage and bandwidth matter.
- **Speed:** Faster read/write operations reduce stutters or frame drops during auto-saves.
- **Battery Efficiency:** Less CPU work prolongs battery life.

## Highlights

- **Automatic Scene Tracking:** Track changes in hierarchical objects and remember if a door is opened or closed, even across scene transitions.
  - **Migration-Friendly:** Seamlessly migrates older save data to newer versions, preventing broken saves when you update your game.
  - **Screenshots:** Each save slot can include a screenshot for a polished user experience.
  - **No-Coding Required:** All major features are accessible through Unity's Inspector, though an API is provided for advanced customization.
- 

## 2. Key Features

- **GC2 Integration:** Save GC2 Variables, including Local Name Variables and Inventory Bag Items.
  - **Lightweight, Non-Blocking:** Minimal overhead on older machines.
  - **Customizable:** Add or remove tracking at runtime, or define exactly which properties to serialize.
  - **Hierarchical Parent Reference:** “Remember” if a prefab was parented under another GameObject or to the scene root.
  - **Unity Native Workflow:** Uses standard Unity components rather than custom editors for easy onboarding.
- 

## 3. Installation & Setup

You can install **Crystal Save** either in a fresh Unity project or by adding it to an existing one.

1st **Download** “Crystal Save” and **GC2 Module** (if you are using Game Creator 2).

2nd **Ensure NuGet Package Manager** is set up in your project (if you plan to address any MemoryPack conflicts manually).

3rd **Import** the “Crystal Save” package into your Unity project.

After importing, you should see the core scripts under:

Assets/Plugins/Arawn/CrystalSave/

### Note

MemoryPack is included by default in “Crystal Save.” If you have no conflicts, **no additional steps** are required to use it.

### 3.1 Fixing the MemoryPack Error

If you encounter an error indicating Unity cannot find **MemoryPack**, do the following:

1st **Delete** any previous **Crystal Save** installation (or install into a new project).

2nd **Delete the Libraries Folder** inside the CrystalSave folder (found under Assets/Plugins/Arawn/CrystalSave).

3rd **Open the NuGet Package Manager:**

- *Menu* → NuGet → Manage NuGet Packages
- Search for **MemoryPack**.
- **Install** the package.

Once re-imported, the error should resolve.

---

## 4. Basic Workflow Scene GameObject

Below is a high-level overview of how **Crystal Save** tracks objects and saves their state.

1st **Add Remember Component and its modules** (like RememberTransform or RememberCollider) to the GameObjects you want to track.

2nd **Enter Play Mode**: On Awake() or OnEnable(), each “Remember” component or SaveablePrefab registers itself with **SaveManager**.

3rd **Save**: When you call SaveManager.Instance.Save(...), the system serializes each tracked object’s data to a binary file.

4th **Load**: On load, the system re-creates the tracked objects (if they’re prefabs) or re-applies the saved data (if they’re scene objects).

### 4.1 Registering an Instantiated Prefab

- **Remember Prefab** is used for runtime-instantiated prefabs.
  - Attach a **Remember Prefab** component to your prefab.
  - The system generates or assigns an internal UniqueID for each instance.
  - Optionally add “Remember Component” and its modules (e.g., RememberTransform) to store position/rotation/scale.
  - **SaveManager** re-instantiates (or re-destroys) these prefabs on load if needed.
-

## 5. Using Crystal Save Components

### Overview

Remember Component **Class name RememberComposite** is a tidy wrapper that *groups* multiple Remember-Me **SaveableComponent** helpers under a single, collapsible header in the Inspector.

### What you can attach

Add as many of these as you need *next to* the composite:

- \* **RememberTransform** – local position / rotation / scale
- \* **RememberRigidbody** – velocity, drag, kinematic flag, gravity, constraints
- \* **RememberAnimator** – layer-0 state hash + normalised time
- \* **RememberGameObject** – active state, tag, layer
- \* **PersistentVisibilityController** – off-screen deactivation rules
- \* Any other custom `SaveableComponent` you made for your game

### Steps to set up

1. Select the GameObject you want to persist.
2. **Add Component** → Crystal Save → **Remember Component** (class name **RememberComposite**).
3. In the newly added Remember Component, click **Add Remember Component** and add the specific helpers you need (e.g. **RememberTransform**).
4. Configure the Helper Module
5. Done

### Under the hood

- \* At edit-time each helper's `OnValidate` checks if it was flagged by the composite and sets `hideFlags` accordingly
- \* Helpers register themselves with the save system using a composite key `<GameObjectGUID>_<ComponentID>`
- \* On load, `SaveManager` feeds the stored bytes back into every helper so the composite reconstructs the object's pose, physics, animator and children.

### Best practices

- \* Put **one** RememberComposite on every dynamic object you intend to save; add helpers to it.
- \* After duplicating an object in the Editor, choose **Generate New IDs** (context menu) to prevent duplicate-GUID warnings.
- \* For runtime-spawned prefabs, combine RememberComposite with `SaveablePrefabFactory.Instantiate()` so every clone gets a fresh `UniqueID`.

### Troubleshooting

**"I can't see the helper scripts."**

They're hidden intentionally. Unlock the Inspector or switch to *Debug* mode to peek.

**"My object is not restoring."**

Ensure the composite is enabled **and** the object carries either `SaveablePrefab` or `UniqueID` so it can be tracked.

## Prefabs and Remember Prefab component (class SaveablePrefab)

### What it Saves & Restores

---

A **Remember Prefab** also known under the **class name `**SaveablePrefab**`** is the Swiss-army knife of Remember Me's save system.

Attach it to any prefab (enemy, UI panel, pickup...) and it automatically captures **everything** a loader needs to respawn that object in exactly the same state—even if it was spawned at runtime or lives in a pool.

### What gets recorded?

---

#### ### 1 · Identity & Links

- \* **`uniqueID`** – the per-instance GUID that ties live objects to bytes in the save file
- \* **`prefabAssetID`** – points back to the original asset so the loader knows which prefab to (re)instantiate
- \* **`Parent / scene flags`** – whether the object is `'DontDestroyOnLoad'`, its parent's ID, and whether that parent is a scene object

#### ### 2 · Transform snapshot

Position, rotation (world or local depending on parent), and local scale are stored on every save

#### ### 3 · Active State & Visibility

- \* ``gameObject.activeSelf`` plus optional per-scene visibility rules.
- \* Off-screen behaviour flags (disable colliders, renderers, etc.) captured via `'PersistentVisibilityController'`.

#### ### 4 · Physics & Animation (via `SaveableComponents`)

If the object carries `'SaveableComponent'`'s—e.g. Remember Me's Rigidbody, Animator, Health, etc.—their byte blobs are stored alongside the prefab data and re-applied on load

#### ### 5 · Runtime Modifications

Edits done **after** the prefab spawned are diffed and saved:

- \* Components **added at runtime** (type + private field values)
- \* **`Mesh`** swaps (`MeshFilter` / `SkinnedMeshRenderer`)
- \* **`Material`** overrides per renderer slot
- \* **`ParticleSystem`** time & playing state, so VFX resume mid-burst
- \* **`Child hierarchy`** changes – child destroyed, re-layered, retagged, or re-activated (**Please note:** Do not rename children of instantiated prefabs. The name of the child is used as Unique ID!)

#### ### 6 · Pooling Support

When the prefab lives in a `'SaveablePrefabPool'`, the pool hook (`'OnBeforeSpawn'`) assigns a fresh `'uniqueID'`, registers the clone, and later soft-unregisters it on despawn so pools don't clutter saves

#### ### 7 · Scene-to-Scene Persistence

`'keepAcrossScenes'` lets a prefab follow the player through level loads by piping it through `PersistentManager.MakePersistent`

### How restore works

---

1. **`PrefabManager`** looks up `'prefabAssetID'`, instantiates the asset, injects the stored `'uniqueID'`, and parents it correctly
2. Transform, active state, visibility settings and physics/animator snapshots are applied.
3. Runtime modifications are replayed (added components, mesh/material swaps, particle rewinds, child edits)
4. `'SaveableComponent'`'s receive their byte blobs.
5. `'SaveManager.OnGameObjectRestored'` fires so gameplay code can react.

## Why it's a “jack of all trades”

-----  
\* **One component** handles identity, registry, persistence, pooling hooks, visibility, runtime diffs, and cross-scene life.

\* Works for **editor-placed objects**, **runtime-spawned clones**, and **objects from pools** with zero extra boiler-plate.

\* Plays nicely with higher-level utilities (`SaveablePrefabFactory`, Targeted Restore, Destroyed-Object Restore) because it provides the canonical data model every other system relies on.

## TL;DR

-----  
Add **SaveablePrefab** to any prefab and Remember Me will save:

\* Where it is, whether it's active, and which asset it came from.

\* Physics, animation and any custom `SaveableComponent` state.

\* Mesh/material/particle overrides and even components you added in code.

On load the object is **re-instantiated and patched** until it matches the exact state it had when the player hit Save—no extra code required.

### Location:

Arawn.CrystalSave.Runtime > SaveablePrefab.cs

For any prefab you instantiate at runtime that you want to persist across loads, use **SaveablePrefab**. In summary:

- Automatically registers prefab instances with the SaveManager.
- Supports `keepAcrossScenes` (equivalent to Unity's `DontDestroyOnLoad` for the prefab instance).
- Has “visibility settings” for different scenes.

When the user saves, any prefab with Remember Prefab Component is serialized. When the user loads, SaveManager re-instantiates or deactivates them as needed.



## 6. Instantiate a Saveable Prefab (Remember Prefab) at runtime

Since Version 1.5.5x this is no longer an issue but before version 1.5.5x

Unity's `Object.Instantiate` has **no awareness of Remember Me's ID, registry, or save-tracking system**. If you spawn a prefab that carries a `SaveablePrefab` component with the plain Unity call, you risk:

- **Stale `uniqueID` field** – Unity copies the editor-time `uniqueID` value into every clone.

Unless a script overwrites it with `Guid.NewGuid()`, all clones share the same logical identity, which breaks save/restore.

- **Missing `PrefabAssetID`** – the clone never registers itself in the `PrefabRegistry`, so later loads can't find which asset to respawn.

- **Skipped registration** – because the object never calls Remember Me's registration pipeline, it remains invisible to `SaveManager`.

The fix is to use **`SaveablePrefabFactory.Instantiate`**, which handles all registration and ID management for you.

### 1. Import the namespace

```
using Arwn.CrystalSave.Runtime;
```

### 2. Call the factory instead of Unity's `Instantiate`

**// A prefab asset that already has a `SaveablePrefab` on it**

```
public GameObject enemyPrefabAsset;
```

```
void SpawnEnemy(Vector3 position)
```

```
{  
  
    // Returns the freshly spawned SaveablePrefab instance  
  
    SaveablePrefab enemy = SaveablePrefabFactory.Instantiate(  
  
        prefabAsset: enemyPrefabAsset,  
  
        position:    position,  
  
        rotation:    Quaternion.identity,  
  
        parent:      null,    // optional  
  
        registerWithSaveSystem: true // default = true  
  
    );  
}
```

## What the factory does under the hood

---

- Instantiates the prefab and parents it if requested.
- Adds a 'SaveablePrefab' component if the asset didn't have one.
- Stores a reference to the original asset ('SetOriginalPrefabAsset') so diff logic works.
- Generates or verifies a 'PrefabAssetID' and ensures the asset is in the central 'PrefabRegistry'.
- Generates a new 'uniqueID' when necessary and registers the clone with the save system.

Because of the registry step the spawned clone can always be recreated during a load: the loader asks the 'PrefabRegistry' for the asset that matches the stored 'PrefabAssetID' and then reconstructs its state.

## Advanced – using pools

If you need high-frequency spawning (bullets, UI pop-ups, etc.) wrap the asset in a **`**`SaveablePrefabPool`**`**.

The pool hands out already initialised clones and calls the same soft-unregister logic when despawning, so pooled objects never bloat your save files.

```
SaveablePrefabPool pool;
```

```
void Awake()
```

```
{  
  
    pool = new SaveablePrefabPool(  
        enemyPrefabAsset.GetComponent<SaveablePrefab>(),  
        initialSize: 10,  
        rememberObjects: true // still saved!  
    );  
}
```

```
void Fire(Vector3 pos, Quaternion rot)
```

```
{  
    var enemy = pool.Spawn(pos, rot);  
}
```

## TL;DR

- **`**Never**`** call ``Object.Instantiate`` on a prefab that contains ``SaveablePrefab``. Since Version 1.5.5x this is no longer an issue but I still recommend the usage of the `SaveablePrefabFactory` to instantiate Prefabs.

- **`**Always**`** spawn through ``SaveablePrefabFactory.Instantiate`` (or a ``SaveablePrefabPool``) so Remember Me can assign fresh IDs, register the object, and guarantee it is saved & restored correctly.

## 7. Targeted Restore – Restoring a single GameObject

\_“Load only what you need, leave the rest of the scene untouched.”\_

Remember Me’s **\*\*Targeted Restore\*\*** lets you pull the saved state of one GameObject back into the live scene without re-loading the scene or touching any other objects.

Typical use-cases:

- \* Re-open a vendor UI and restore its inventory
- \* Snap the player back to the last checkpoint
- \* Respawn a boss arena door that was opened earlier
- \* **\*\*Debug\*\*** – compare a live object against a past save without restarting

**The feature is delivered by two overloads on `SaveManager`:**

```
// 1) Pass the live object you already have
public async void RestoreSingleGameObject(GameObject target, int slotNumber);

// 2) Pass the UniqueID (string) of an object already in the scene
public async void RestoreSingleGameObject(string uniqueID, int slotNumber);
```

### How it works under the hood

-----

1. The chosen save slot is opened and deserialised (cloud or local, async-safe).
2. The helper finds the object in the current scene – either from the `GameObject` reference you passed or via its `uniqueID`.
3. Only the bytes that belong to that object are applied:
  - \* active state
  - \* transform / prefab snapshot
  - \* Rigidbody & Animator snapshots if present
  - \* per-component data gathered by `ComponentManager`
4. `SaveManager.OnGameObjectRestored(uniqueID)` is fired so UI or gameplay code can react.

## Calling the method – examples

---

### ### You already have the object reference

```
// Restore the player from slot 1
SaveManager.Instance.RestoreSingleGameObject(playerGO, 1);
```

### ### You only know the UniqueID (Inspector Debug view shows the UniqueID in the UniqueID component)

```
const string bossDoorID = "Door_BossRoom_003";
SaveManager.Instance.RestoreSingleGameObject(bossDoorID, 3);
```

### ### Instant re-apply from the current in-memory save

If you have already loaded a save and want to *\*re-apply\** it (e.g. a “reset puzzle” button) call the overload without a slot number:

```
SaveManager.Instance.RestoreSingleGameObject(puzzleGO);
```

## Practical tips

---

- \* Ensure the target object exists in the scene; if it was destroyed, use `RestoreDestroyedGameObject` instead.
- \* The call is `async void`; heavy objects may take one frame to snap into their final state — wait a frame before starting cut-scenes.
- \* Hook into `SaveManager.OnGameObjectRestored` to run VFX or sounds when a restore finishes.

## TL;DR

---

`RestoreSingleGameObject` is your precision tool: pick one object, pick a slot, and Remember Me reapplies just that slice of the save file. No loading screens, no side-effects — perfect for mid-game reset buttons and dynamic UIs.

## 8. Restoring Destroyed GameObjects

Where **Targeted Restore** re-applies data to an *existing* object, the **Destroyed-Object Restore** feature brings back *objects* that were completely removed from the scene (e.g. breakable crates, pickups or temporary enemies).

### Why you need it

-----

- \* Undo a player's mistake (e.g. respawn a vendor they accidentally killed).
- \* Reset a puzzle room without re-loading the level.
- \* Restore critical interactables after a hot-reload in the editor.

### Public API

-----

```
// Restore one previously-destroyed object
public void RestoreDestroyedGameObject(string uniqueID);

// Restore all destroyed objects that still have a prefab mapping
public void RestoreAllDestroyedGameObjects();
```

### How it works under the hood

-----

1. When a tracked object is destroyed, `SaveManager.RegisterDestroyedGameObject` is called. It:
  - \* Captures the object's last component snapshot.
  - \* Adds its `uniqueID` to `destroyedGameObjectIDs`.
  - \* Removes the live reference from tracking dictionaries.
2. During **Save**, that list is written to `SaveData.DestroyedGameObjects` along with the captured component bytes.
3. `RestoreDestroyedGameObject` checks the ID, fetches the original prefab from `SceneObjectRegistry`, `Instantiate`'s a fresh clone, sets the same `uniqueID`, re-applies the stored component data, and re-registers the object for future saves.
4. Finally it fires `OnGameObjectRestored` so gameplay code can react.

## Usage examples

-----

### 1) Respawn a single breakable crate

```
const string crateID = "Crate_TutorialRoom_05";

// Player hit the "Reset Crate" button:
SaveManager.Instance.RestoreDestroyedGameObject(crateID);
```

### 2) Reset an arena between waves

```
// Between waves, bring back *every* destroyed prop
SaveManager.Instance.RestoreAllDestroyedGameObjects();
```

### 3) Safety check before restoring

```
string npcID = "Vendor_Myrna";

// Only respawn if she is actually gone
if (SaveManager.Instance.IsGameObjectDestroyed(npcID))
{
    SaveManager.Instance.RestoreDestroyedGameObject(npcID);
}
//..
```

## Practical tips

-----

- \* The **prefab** must be registered in **SceneObjectRegistry** at build-time; otherwise the restore call logs a warning and does nothing.
- \* Component data is only available if the object was **tracked** when it was destroyed (i.e. had a `RememberGameObject` with `SaveDestroy = true`). If not, the prefab spawns in its default state.
- \* For large batches (`RestoreAllDestroyedGameObjects`) the method loops over **base IDs** so variants like `"Crate_001_Shards"` are grouped and respawned together.

## TL;DR

-----

`RestoreDestroyedGameObject` brings **deleted objects** back to life, re-using their saved component data so they look exactly as when they were destroyed. Pair it with `IsGameObjectDestroyed` to build smart respawn or level-reset systems without reloading your scene.

## 9. Scene Transitions & Keep Across Scenes

If you want to keep a `GameObject` (or prefab instance) across scene transitions, you can set:

```
// Within your SaveableComponent or SaveablePrefab
keepAcrossScenes = true;
Internally, Crystal Save calls:
```

```
DontDestroyOnLoad(gameObject);
```

### Limitation: Child Objects

- If a child object is also using `KeepAcrossScenes`, it must be the **parent** that actually has `keepAcrossScenes = true`. Unity does not allow a child to be in a different scene from its parent.
  - For example, if a door is parented under another object, enable "Keep Across Scenes" on the parent, not the door (the child).
-

## 10. User Settings Manager

### Why it exists

UserSettingsManager is a **lightweight, opt-in** service that keeps all player-configurable options—resolution, audio levels, language, HDRP/URP effect toggles—inside a single JSON file. It applies the choices automatically on every scene load, so you never sprinkle quality-setup code around your project.

---

### When to add it (and when not to)

- **Optional by design** – Crystal Save’s core save flow works fine without it.
  - **Use it if** you ship any kind of options menu or want one-click cloud sync for graphics & audio preferences.
  - **Skip it if** your game sticks with Unity’s default quality settings or you already have a custom settings pipeline.
- 

### Installing in the scene

1st Drag the **UserSettingsManager** prefab (or add the component) into a “Bootstrap” scene or a persistent loader prefab.

2nd Make sure you have **only one** copy in the whole project. The first instance survives scene changes via DontDestroyOnLoad; duplicates self-destruct.

3rd (Optional) In the inspector wire references for:

- Main Camera
  - Global Volume or PPv2 PostProcessVolume
  - Master AudioMixer
- If you leave these blank the manager finds them at runtime.
- 

### Runtime life-cycle in plain English

- **Awake** – lazily claims the singleton, marks itself persistent.
  - **Start** – tries to read *UserSettings.json* from Application.persistentDataPath; if missing, spins up defaults.
  - **ApplySettings()** – pushes everything into Unity’s live systems, then listens for SceneManager.sceneLoaded to re-apply on each new level.
  - **SaveSettings()** – serialises the in-memory struct back to *UserSettings.json*. Call this from your “Apply” or “Save & Quit” buttons.
-



## Hooking it into an options menu (coder view)

- Read the current values directly for default slider/checkbox states:  
var volume = UserSettingsManager.Instance.CurrentSettings.MusicVolume;
- On UI interaction set the field, then either:  
Call ApplySettings() for instant preview, **or**
- Call SaveSettings() to persist and apply in one go.
- Example snippet for a resolution dropdown:

```
public void OnResolutionChanged(int width, int height, bool fullscreen)
{
    var mgr = UserSettingsManager.Instance;
    mgr.UpdateResolution(width, height, fullscreen);
    mgr.SaveSettings();
}
```

---

## Day-to-day API cheat-sheet

- CurrentSettings – plain-old-data struct you can read & write.
- ApplySettings() – push in-memory values to Unity, no file I/O.
- SaveSettings() – write JSON **and** call ApplySettings().
- LoadSettings() – force a fresh disk read (rarely needed).
- High-level helpers such as UpdateVSync, UpdateTargetFrameRate, SetAudioMixerReference streamline the most common tweaks.

---

## Tips for technical designers

- **Instant iteration:** change a Boolean or slider in the component while in Play Mode, hit **Apply Settings** in the inspector, see it ripple through every loaded object.
- **Locale testing:** swap the LocaleCode, press Apply, all localisation tables update on the spot.
- **Profile fine-tuning:** in HDRP or URP you can enable/disable Bloom, AO, DOF live without touching volume assets—ideal for quick profile comparisons.

---

## Extending with custom settings

1st Add new fields to the UserSettingsData struct (e.g., public float Gamma;).

2nd Update ApplySettings() so the new field actually drives something—like a colour-grading exposure on your post-process volume.

3rd Recompile, run once, and the new value appears in your JSON file with its default.

---

## Best practice checklist

- Place the component **once** in your boot path.
- Pair it early with SaveManager: both default to execution order -100, ensuring they initialise before gameplay scripts.
- Never use raw PlayerPrefs for user options—pipe everything through UserSettingsManager so local JSON and cloud sync stay in sync.
- Call SaveSettings() on a safe event (Apply button, “Quit Game”, or OnApplicationQuit) so last-second tweaks aren’t lost.

Drop it in when you need player-friendly options; leave it out if you don’t. Either way, Crystal Save remains plug-and-play.

## 11. Working With GC2 Integration

**Crystal Save** offers a **GC2 Module** so you can:

- **Save GC2 Variables:** If you have a GC2 Local Variable on a GameObject, attach the Remember GC2 Local Name Variables component to store its value.
- **Save GC2 Inventory (Bags):** Attach Remember GC2 Inventory to your chest or other item container.
- And more...

**Note:** GC2's own save system uses a JSON-based approach, which can be slower or more prone to large file sizes.

**Crystal Save** is fully binary-serialized, resulting in smaller save files and faster load times, especially on mobile.

---

## 12. Save Slots and Screenshots

**Crystal Save** manages multiple **Save Slots**—the number is configured in **SaveSettings**. For example, you might have 10 or more slots, each containing:

- A **Binary** data file of your game state
- Optionally a **Screenshot** representing your game's visuals at the time of saving

Use:

```
SaveManager.Instance.Save(slotNumber);  
SaveManager.Instance.Load(slotNumber, restoreLastActiveScene: true);
```

**Screenshots** are automatically captured if enabled in the **SaveSettings**.

---

## 13. Auto-Migration & Versioning

**Crystal Save** has a built-in **Migration Manager** to ensure older save files remain compatible as you update your game.

**Key steps:**

1st **Increment Version** in Crystal Save > Settings (e.g., from 1.0.0 to 2.0.0).

2nd Create **Migration Actions** via the Migration Window. For example, you can create an action to update a certain Transform position or unify lighting changes.

3rd The system automatically upgrades old save data to the new version upon load—your players don't lose their progress!

---

## 14. Limitations & Caveats

### 1st DontDestroyOnLoad & Hierarchies

- If you enable keepAcrossScenes, it effectively calls DontDestroyOnLoad. Nested hierarchies must be consistent: a child cannot be set to remain across scenes if its parent is not.

### 1st Live Enable/Disable of Saveable Components

- By default, if you destroy a GameObject or disable the saving component, the system will unregister it automatically (on OnDestroy() or OnDisable()). However, dynamically re-registering or deregistering during gameplay is possible and demonstrated in the GameObject Pool mechanism.

### 1st Binary Data

- Because it's not text-based, you can't easily merge or manually edit saves. This is a design trade-off for performance and size benefits.

### 1st Scene List

- Visibility toggles in **Crystal Save** rely on the "Scene List" you define in the settings. Make sure you add all relevant scenes here for correct enabling/disabling of colliders, renderers, etc.
-

## 15. API References

### SaveManager – Core Public API

---

Below is a quick reference for the **most frequently used** public methods on **SaveManager**. SaveManager is a Singleton and auto-adds itself in every scene on Awake.

All parameters are shown with their default values where applicable so you can copy-paste the call straight into your code.

#### Save & Load

---

```
`void Save(int slotNumber = 1, string lastActiveScene = null)`
```

Write the current game state to *\*slotNumber\**.  
If you pass *'lastActiveScene'*, that scene name is stored in the metadata

```
`void Load(int slotNumber = 1, bool restoreLastActiveScene = false,  
    bool loadAsync = false, bool allowSceneActivation = true,  
    CancellationToken cancellationToken = default)`
```

Starts a coroutine that deserialises the chosen slot and rebuilds the scene. Set *'loadAsync'/'allowSceneActivation'* to integrate with custom loading screen

```
`Task<LoadResult> LoadSaveSlotAsync(int slotNumber = 1,  
    bool restoreLastActiveScene = false,  
    bool loadAsync = false,  
    bool allowSceneActivation = true,  
    TimeSpan? timeout = null,  
    CancellationToken token = default)`
```

Await-able wrapper that completes when the internal load coroutine finishes or times out. Handy for menu flows and integration tests

```
`Task<SaveData> LoadSaveDataForSlotAsync(int slotNumber)`
```

Fetches the raw *'SaveData'* blob **without** touching the scene – perfect for displaying slot summaries or thumbnails in a save-select UI

## Slot Management

-----

**`void Delete(int slotNumber)`**

Removes a slot (and its screenshot mirror if present).

Triggers ``OnDeleteCompleted`` / ``OnDeleteFailed`` events

-----

**`void RenameSlot(int slotNumber, string newName)`**

Rename directly from the main thread. Fires ``OnRenameSlotCompleted`` /

``OnRenameSlotFailed`` so your UI can refresh

-----

**`Task<bool> RenameSaveSlotAsync(int slotNumber, string newName)`**

Safe to call from background threads; internally enqueues the work back to the main thread and awaits the result

## Quality-of-life helpers

-----

**`List<SaveSlot> GetSaveSlots()`** – array-style access to all slots

**`SaveSlot GetLatestSaveSlot()`** – returns the most recently saved slot

**`bool HasSave()`** / **`HasSaveAt(int slot)`** / **`HasSaveAfterDate(date)`** / **`HasSaveInScene(scene)`** – quick checks for UI gating

## Events (selected)

-----

\* ``OnSaveCompleted``, ``OnSaveFailed``

\* ``OnLoadCompleted``, ``OnLoadFailed``

\* ``OnDeleteCompleted``, ``OnRenameSlotCompleted``

\* ``OnSceneLoadStarted``, ``OnSceneLoadProgress``, ``OnSceneActivationRequested``

These let you hook UI feedback, analytics or additional gameplay logic into any operation.

## Usage tips

-----

\* Always await ``LoadSaveSlotAsync`` or listen to ``OnLoadCompleted`` before starting gameplay logic that depends on restored objects.

\* Call ``Save()`` from the main thread (e.g. via button or auto-save coroutine). It already spins heavy work off to background threads.

\* If you support both local and cloud saves, check ``SaveSettings.enableCloudSave`` first so you can display the right messaging while a remote upload is in progress.

---

## 16. Conclusion

**Crystal Save** is designed to be a plug-and-play Save System for Unity, especially well-suited for medium and large sized projects or mobile platforms that demand efficient, compact saves. By attaching the appropriate “Remember” components and letting the **SaveManager** handle the data flow, you can reliably persist and restore game states—even across major version changes—without writing custom serialization code.

### Need More Info?

- Check out the included **Demo Scenes** under Assets/Plugins/Arawn/CrystalSave/Modules/Unity/Demos/.
- For advanced GC2 usage, refer to Remember GC2 Inventory, Remember GC2 Local Name Variables, and the **GC2 Module** documentation.
- If you have further questions or encounter issues, consult the MemoryPack Error Fix steps or contact support.

Happy saving and loading! If you maintain these guidelines and remember the limitations, **Crystal Save** should provide a smooth and robust saving experience for your Unity game.

## 17. Credits

This asset requires MemoryPack 1.21.4 (MIT) and Unity Editor Coroutines. Both can be auto-installed the first time you import the asset.

MemoryPack 1.21.4 – Binary serialization library  
Copyright © 2022–2025 Cysharp, Inc.  
MIT License • <https://github.com/Cysharp/MemoryPack>

Used by Crystal Save as its default serializer.  
No files from MemoryPack are redistributed with this asset;  
the DLLs are fetched automatically from nuget.org when the user  
opts in during the first import.

BouncyCastle.Cryptography 2.6.1  
(© The Legion of the Bouncy Castle — MIT)  
Licence → <https://github.com/bcgit/bc-csharp/blob/master/LICENSE.md>

Used only when the active Build Target is WebGL or Linux (AES-GCM / SHA-3 for browser builds).  
No files from Bouncy Castle are redistributed with this asset;  
the DLL is fetched automatically from nuget.org when the user  
opts in during the first import.